AutoEIT — EIT Audio Transcription Pipeline

GSoC 2026 | HumanAI Foundation (NIU + University of Alabama)

Two-stage pipeline:

- **Stage 1 — Speaker Separator:** WhisperX `large-v3` + pyannote diarization separates the two voices (experimenter playback + learner response) and writes `output/<id>/transcript.txt` per file
- **Stage 2 — AutoEIT Transcriber:** reads each transcript, finds the English/Spanish boundary, fuzzy-matches segments to the 30 EIT stimuli, and fills column C of the Excel template

Tunable thresholds at the top of Stage 2:

- `PAIR_THRESHOLD = 55` — similarity required to keep two consecutive segments as a stimulus/response pair
- `STIMULUS_MATCH_THRESHOLD = 45` — minimum match score before a lone segment is kept

⚡ **Set runtime to T4 GPU** → *Runtime → Change runtime type → T4 GPU*

🔑 **HuggingFace token required** for diarization. Accept the license at pyannote/speaker-diarization-3.1 then add as `HF_TOKEN` in Colab Secrets.

Section 1 — Install Dependencies

Run once. If the runtime restarts after the torch upgrade, skip to Section 2.

```
# # ── 1A: Install dependencies ──────────────────────────────────────────
# !pip install whisperx pydub -q
# !pip install rapidfuzz openpyxl langdetect -q
# !apt-get install -y ffmpeg -q
# !pip install torch torchvision torchaudio --upgrade -q

# print("✅ Dependencies installed")
# print("⚠️ Please restart the runtime (Runtime → Restart runtime) to apply the changes.")
```

Section 2 — Folder Setup & File Upload

Upload your files after running this cell:

- 4 × `.mp3` files → `./input/`
- `AutoEIT Sample Audio for Transcribing.xlsx` → `./` (root)

```
!mkdir -p "./input" "./output"
```

```
# Verify uploads
from pathlib import Path
mp3s  = sorted(Path("./input").glob("*.mp3"))
excel = Path("./AutoEIT Sample Audio for Transcribing.xlsx")
print(f"MP3 files : {len(mp3s)}")
for f in mp3s:
    print(f"  ✅  {f.name}  ({f.stat().st_size/1e6:.1f} MB)")
print(f"\nExcel : {'✅' if excel.exists() else '❌ MISSING'}  {excel.name}")
```

```
MP3 files : 4
  ✅  038010_EIT-2A.mp3  (9.3 MB)
  ✅  038011_EIT-1A.mp3  (9.6 MB)
  ✅  038012_EIT-2A.mp3  (19.0 MB)
  ✅  038015_EIT-1A.mp3  (9.0 MB)

Excel : ✅  AutoEIT Sample Audio for Transcribing.xlsx
```

Section 3 — Stage 1: Speaker Separation & Transcription

Runs WhisperX + pyannote on every `.mp3` in `./input/`. For each file produces:

- `output/<filename>/transcript.txt` — timestamped, speaker-labelled transcript
- `output/<filename>/speaker_1.mp3` / `speaker_2.mp3` — isolated audio per speaker

Expected time: ~~8–12 min per file on T4 GPU~~ (40 min for all 4)

3.1 — Imports & compatibility patches

These patches **must run before any WhisperX import**. They fix three known incompatibilities:

1. **PyTorch 2.6+** — strict `weights_only` loading breaks model checkpoints; patched to always pass `False`
2. **HuggingFace Hub** — deprecated keyword args (`use_auth_token`, `resume_download`) cause errors; stripped at call time
3. **torchaudio** — missing API methods in some versions; shimmed with `soundfile` equivalents

```
"""
Speaker Separator using WhisperX  — Batch Folder Mode
──────────────────────────────────────────────────────
Usage:
    python speaker_separator.py --folder ./my_audio_files/
    python speaker_separator.py --folder ./my_audio_files/ --output ./results/

Requirements:
    pip install whisperx pydub
"""

import os
import sys
import argparse
import warnings
warnings.filterwarnings("ignore")

# ═══════════════════════════════════════════════════════════════════════
# ── MASTER PATCH BLOCK (MUST RUN BEFORE WHISPERX) ──────────────────
import torch
import torchaudio
import huggingface_hub
import soundfile as sf

# 0. PyTorch 2.6+ Strict Loading Bypass (Double-Run Proof!)
if not getattr(torch, '_load_is_patched', False):
    _orig_torch_load = torch.load
    def _patched_torch_load(*args, **kwargs):
        kwargs['weights_only'] = False
        return _orig_torch_load(*args, **kwargs)
    torch.load = _patched_torch_load
    torch._load_is_patched = True

_orig_hf_hub_download = huggingface_hub.hf_hub_download
def _patched_hf_hub_download(*args, **kwargs):
    if 'use_auth_token' in kwargs: kwargs['token'] = kwargs.pop('use_auth_token')
    kwargs.pop('resume_download', None)
    kwargs.pop('force_filename', None)
    kwargs.pop('local_dir_use_symlinks', None)
    try: return _orig_hf_hub_download(*args, **kwargs)
    except Exception as e:
        if "custom.py" in str(kwargs.get('filename', '')) and ("404" in str(e) or "EntryNotFound" in type(e).__name__):
            raise ValueError("File not found")
        raise e
huggingface_hub.hf_hub_download = _patched_hf_hub_download

_orig_model_info = huggingface_hub.model_info
def _patched_model_info(*args, **kwargs):
    if 'use_auth_token' in kwargs: kwargs['token'] = kwargs.pop('use_auth_token')
    return _orig_model_info(*args, **kwargs)
huggingface_hub.model_info = _patched_model_info

class DummyAudioMetaData:
    def __init__(self, sr, frames, ch):
        self.sample_rate = sr; self.num_frames = frames; self.num_channels = ch
def _patched_info(uri, **kwargs):
    info = sf.info(uri)
    return DummyAudioMetaData(info.samplerate, info.frames, info.channels)
def _patched_load(uri, **kwargs):
    data, sr = sf.read(uri, always_2d=True, dtype='float32')
    return torch.from_numpy(data).t(), sr
def _patched_save(filepath, src, sample_rate, **kwargs):
    sf.write(filepath, src.t().numpy(), sample_rate)
```

```

if not hasattr(torchaudio, 'AudioMetaData'): torchaudio.AudioMetaData = DummyAudioMetaData
if not hasattr(torchaudio, 'list_audio_backends'): torchaudio.list_audio_backends = lambda: ['soundfile', 'sox_io']
if not hasattr(torchaudio, 'info'): torchaudio.info = _patched_info
if not hasattr(torchaudio, 'load'): torchaudio.load = _patched_load
if not hasattr(torchaudio, 'save'): torchaudio.save = _patched_save
# ═══════════════════════════════════════════════════════════════════════
```

3.2 — `from whisperx.diarize import DiarizationPipeline`

Imported here (after the patches) so the compatibility fixes above are already active.

```
from whisperx.diarize import DiarizationPipeline
```

3.3 — HuggingFace token loader

Tries three sources in order: Colab Secrets (`HF_TOKEN`) → environment variable → hardcoded fallback. Required to download the pyannote diarization model.

```

# ─── HF TOKEN — tries Colab secrets first, then env var, then hardcoded ──────
def load_hf_token():
    try:
        from google.colab import userdata
        token = userdata.get('HF_TOKEN')
        if token:
            print(f"  ✅ HF token loaded from Colab secrets: {token[:8]}...")
            return token
    except Exception:
        pass
    token = os.environ.get('HF_TOKEN', '')
    if token:
        print(f"  ✅ HF token loaded from environment: {token[:8]}...")
        return token
    # Fallback — hardcode here if needed
    hardcoded = ""   # ← paste token here as last resort
    if hardcoded:
        print(f"  ✅ HF token loaded from hardcoded value: {hardcoded[:8]}...")
        return hardcoded
    print("  ❌ HF_TOKEN not found in Colab secrets, environment, or hardcoded value.")
    return ""

```

3.4 — Config & helper functions

Key settings you can edit:

- `WHISPER_MODEL` — `"large-v3"` for best accuracy, `"large"` as fallback
- `INPUT_FOLDER` / `OUTPUT_DIR` — paths
- `TARGET_DBFS` — normalisation level
- `LANGUAGE` — forced to `"es"` (Spanish)
- `INITIAL_PROMPT` — the L2-aware transcription prompt passed to Whisper on every segment

Helper functions below (`color`, `format_time`, `print_segment`, etc.) handle coloured console output only.

```

# ─── CONFIG ──────────────────────────────────────────────────────────────

INPUT_FOLDER  = "./audio_files"
OUTPUT_DIR    = "speaker_output"
# WHISPER_MODEL = "large"
WHISPER_MODEL = "large-v3"
TARGET_DBFS   = -20.0
LANGUAGE      = "es"

INITIAL_PROMPT = (
"""
    Un estudiante de español L2 repite una frase.
Transcribe exactamente lo que dice.
Incluye errores gramaticales, disfluencias (eh, mm, um),
falsos inicios con guión (ej: la- las), y pausas con '...'.
Si hay una pausa demasiado larga, probablemente indica el inicio de la siguiente oración.
```

```
No corrijas ni normalices el texto.
No traduzcas ninguna frase en español al inglés."""
)

# ──────────────────────────────────────────────────────────────────────────

COLORS = {
    "Speaker 1": "\033[94m",
    "Speaker 2": "\033[92m",
    "Speaker 3": "\033[93m",
    "Speaker 4": "\033[95m",
    "reset":     "\033[0m",
    "bold":      "\033[1m",
    "dim":       "\033[2m",
    "header":    "\033[96m",
    "green":     "\033[92m",
    "red":       "\033[91m",
    "yellow":    "\033[93m",
}

def color(text, *keys):
    codes = "".join(COLORS.get(k, "") for k in keys)
    return f"{codes}{text}{COLORS['reset']}"

def format_time(seconds):
    m, s = divmod(int(seconds), 60)
    return f"{m:02d}:{s:02d}"

def print_segment(speaker, start, end, text, index):
    spk_color = COLORS.get(speaker, "\033[97m")
    ts  = color(f"[{format_time(start)} → {format_time(end)}]", "dim")
    spk = f"{spk_color}{COLORS['bold']}{speaker:<10}{COLORS['reset']}"
    idx = color(f"#{index:03d}", "dim")
    print(f"  {idx}  {ts}  {spk}  {text}")

def print_header(title):
    width = 70
    print("\n" + color("─" * width, "header"))
    print(color(f"  {title}", "header", "bold"))
    print(color("─" * width, "header"))

def print_banner(title):
    width = 70
    print("\n" + color("=" * width, "bold"))
    print(color(f"  {title}", "bold"))
    print(color("=" * width, "bold"))

def check_requirements():
    missing = []
    for pkg, imp in [("whisperx", "whisperx"), ("pydub", "pydub")]:
        try:
            __import__(imp)
        except ImportError:
            missing.append(pkg)
    if missing:
        print(color(f"\n❌ Missing packages. Run:\n\n   pip install {' '.join(missing)}\n", "bold"))
        sys.exit(1)

```

3.5 — `process_file()` — per-file pipeline

For a single mp3 file:

1. Load + normalise audio to `TARGET_DBFS`
2. Transcribe with WhisperX
3. Align word timestamps
4. Diarise to exactly 2 speakers
5. Write `transcript.txt` + per-speaker `.mp3`

```


def process_file(mp3_path, file_output_dir, whisper_model, align_model,
                 metadata, diarize_model, device, token):
    from pydub import AudioSegment
    from collections import defaultdict
    import whisperx

```

```
    os.makedirs(file_output_dir, exist_ok=True)
    wav_path = os.path.join(file_output_dir, "_working.wav")

    print(color("  ⏳ Loading audio…", "dim"))
    audio_pydub    = AudioSegment.from_mp3(mp3_path)
    total_duration = len(audio_pydub) / 1000
    print(f"     ✅ Loaded  ({total_duration:.1f}s)")

    print(color("  ⏳ Normalizing…", "dim"))
    delta       = TARGET_DBFS - audio_pydub.dBFS
    audio_pydub = audio_pydub.apply_gain(delta)
    print(f"     ✅ Normalized  ({delta:+.1f} dB  →  {audio_pydub.dBFS:.1f} dBFS)")

    audio_pydub.export(wav_path, format="wav")
    audio_array = whisperx.load_audio(wav_path)

    print(color("  ⏳ Transcribing…", "dim"))
    transcribe_kwargs = {"batch_size": 16}
    if LANGUAGE:
        transcribe_kwargs["language"] = LANGUAGE
    result   = whisper_model.transcribe(audio_array, **transcribe_kwargs)
    language = result.get("language", LANGUAGE or "en")
    print(f"     ✅ Transcription done  (language: {language})")

    print(color("  ⏳ Aligning word timestamps…", "dim"))
    if metadata.get("language") != language:
        print(f"     ⚠️  Language mismatch ({metadata.get('language')} → {language}), reloading align model…")
        align_model, metadata = whisperx.load_align_model(language_code=language, device=device)
        metadata["language"] = language
    result = whisperx.align(result["segments"], align_model, metadata, audio_array, device)
    print("     ✅ Alignment done")

    print(color("  ⏳ Diarizing speakers…", "dim"))
    diarize_segs = diarize_model(audio_array, min_speakers=2, max_speakers=2)
    result       = whisperx.assign_word_speakers(diarize_segs, result)
    print("     ✅ Diarization done")

    raw_speakers = sorted({
        seg.get("speaker", "UNKNOWN")
        for seg in result["segments"] if seg.get("speaker")
    })
    speaker_map = {raw: f"Speaker {i+1}" for i, raw in enumerate(raw_speakers)}

    segments = []
    for seg in result["segments"]:
        raw = seg.get("speaker", "UNKNOWN")
        segments.append({
            "speaker":  speaker_map.get(raw, "Unknown"),
            "start":    seg["start"],
            "end":      seg["end"],
            "start_ms": int(seg["start"] * 1000),
            "end_ms":   int(seg["end"]   * 1000),
            "text":     seg["text"].strip(),
        })

    print_header(f"  SEGMENTS  ({len(segments)} total)")
    for i, seg in enumerate(segments, 1):
        print_segment(seg["speaker"], seg["start"], seg["end"], seg["text"], i)

    print_header("  SPEAKER SUMMARY")
    spk_stats = defaultdict(lambda: {"count": 0, "duration": 0.0})
    for seg in segments:
        spk_stats[seg["speaker"]]["count"]    += 1
        spk_stats[seg["speaker"]]["duration"] += seg["end"] - seg["start"]

    for spk, stats in sorted(spk_stats.items()):
        spk_color = COLORS.get(spk, "\033[97m")
        bar_len   = int(stats["duration"] / total_duration * 40)
        bar       = "▓" * bar_len + "▒" * (40 - bar_len)
        print(f"\n  {spk_color}{COLORS['bold']}{spk}{COLORS['reset']}")
        print(f"    Segments : {stats['count']}")
        print(f"    Duration : {stats['duration']:.1f}s  "
              f"({stats['duration'] / total_duration * 100:.1f}%)")
        print(f"    {spk_color}{bar}{COLORS['reset']}")

    transcript_path = os.path.join(file_output_dir, "transcript.txt")
    with open(transcript_path, "w", encoding="utf-8") as f:
        f.write("=" * 70 + "\n")
        f.write("        SPEAKER-SEPARATED TRANSCRIPT (WhisperX)\n")
        f.write(f"        Source: {os.path.basename(mp3_path)}\n")
```

```
        f.write("=" * 70 + "\n")

        current_speaker = None
        for seg in segments:
            if seg["speaker"] != current_speaker:
                current_speaker = seg["speaker"]
                f.write(f"\n{'—'*70}\n{current_speaker}:\n")
            f.write(f"  [{format_time(seg['start'])} → {format_time(seg['end'])}]  {seg['text']}\n")

        f.write(f"\n{'='*70}\nSPEAKER STATS\n{'='*70}\n")
        for spk, stats in sorted(spk_stats.items()):
            f.write(f"  {spk}: {stats['count']} segments, {stats['duration']:.1f}s "
                    f"({stats['duration']/total_duration*100:.1f}%)\n")

    print_header("  SAVING FILES")
    print(f"\n  ✅ Transcript  →  {transcript_path}")

    speaker_audio = {}
    for seg in segments:
        spk   = seg["speaker"]
        chunk = audio_pydub[seg["start_ms"]:seg["end_ms"]]
        if spk not in speaker_audio:
            speaker_audio[spk] = AudioSegment.silent(duration=0)
        speaker_audio[spk] += chunk

    for spk, clip in sorted(speaker_audio.items()):
        fname     = f"{spk.replace(' ', '_').lower()}.mp3"
        clip_path = os.path.join(file_output_dir, fname)
        clip.export(clip_path, format="mp3")
        print(f"  ✅ {spk} audio  →  {clip_path}  ({len(clip)/1000:.1f}s)")

    os.remove(wav_path)
    return len(segments)

```

3.6 — `main()` — run on all files ▶️

Loads all three models **once** (Whisper, alignment, diarization), then calls `process_file()` for every `.mp3` in `./input/`. **Run this cell to start Stage 1.**

```
def main():
    check_requirements()
    input_folder = "./input"
    output_root  = "./output"

    import torch
    import whisperx

    # ── Token — Colab → env var → hardcoded ──────────────────────────────
    token = load_hf_token()
    if not token:
        sys.exit(1)

    if not os.path.isdir(input_folder):
        print(color(f"\n❌ Folder not found: {input_folder}\n", "bold"))
        sys.exit(1)

    mp3_files = sorted([
        os.path.join(input_folder, f)
        for f in os.listdir(input_folder)
        if f.lower().endswith(".mp3")
    ])

    if not mp3_files:
        print(color(f"\n⚠️  No MP3 files found in: {input_folder}\n", "yellow"))
        sys.exit(0)

    os.makedirs(output_root, exist_ok=True)
    device  = "cuda" if torch.cuda.is_available() else "cpu"
    compute = "float16" if device == "cuda" else "int8"

    print_banner("SPEAKER SEPARATOR — WhisperX  (Batch Mode)")
    print(f"\n  📂 Input folder : {input_folder}")
    print(f"  📁 Output root  : {output_root}/")
    print(f"  🎵 Files found  : {len(mp3_files)}")
    print(f"  🖥️  Device       : {device.upper()}  |  Model : {WHISPER_MODEL}")
    print(f"  🌐 Language     : {LANGUAGE or 'auto-detect'}")
    print(f"\n  Files to process:")
```

```
    for i, f in enumerate(mp3_files, 1):
        print(f"    {i}. {os.path.basename(f)}")

    print_header("LOADING MODELS  (once for all files)")

    print(color("\n  ⏳ Loading Whisper model…", "dim"))
    asr_options   = {"initial_prompt": INITIAL_PROMPT} if INITIAL_PROMPT else {}
    whisper_model = whisperx.load_model(WHISPER_MODEL, device, compute_type=compute,
                                        asr_options=asr_options)
    if INITIAL_PROMPT:
        print(f"     📝 Initial prompt active ({len(INITIAL_PROMPT)} chars)")
    print("     ✅ Whisper ready")

    print(color("  ⏳ Loading alignment model…", "dim"))
    align_lang            = LANGUAGE if LANGUAGE else "en"
    align_model, metadata = whisperx.load_align_model(language_code=align_lang, device=device)
    metadata["language"]  = align_lang
    print(f"     ✅ Alignment model ready  (language: {align_lang})")

    print(color("  ⏳ Loading diarization pipeline…", "dim"))
    diarize_model = DiarizationPipeline(token=token, device=device)
    print("     ✅ Diarization pipeline ready")

    results_summary = []

    for i, mp3_path in enumerate(mp3_files, 1):
        filename    = os.path.splitext(os.path.basename(mp3_path))[0]
        file_output = os.path.join(output_root, filename)

        print_banner(f"FILE {i}/{len(mp3_files)}  —  {os.path.basename(mp3_path)}")

        try:
            n_segments = process_file(
                mp3_path, file_output, whisper_model,
                align_model, metadata, diarize_model, device, token
            )
            results_summary.append((os.path.basename(mp3_path), "✅", n_segments, file_output))
            print(color(f"\n  ✅ Done  →  {file_output}/", "green", "bold"))

        except Exception as e:
            results_summary.append((os.path.basename(mp3_path), "❌", 0, str(e)))
            print(color(f"\n  ❌ Failed: {e}", "red", "bold"))

    print_banner("BATCH COMPLETE 🎉")
    print(f"\n  {'FILE':<35}  {'STATUS':<6}  SEGMENTS / OUTPUT")
    print(f"  {'-'*35}  {'-'*6}  {'-'*30}")
    for fname, status, n, out in results_summary:
        info = f"{n} segments  →  {out}" if status == "✅" else out
        print(f"  {fname:<35}  {status:<6}  {info}")

    passed = sum(1 for _, s, _, _ in results_summary if s == "✅")
    print(f"\n  {passed}/{len(mp3_files)} files processed successfully.\n")


if __name__ == "__main__":
    main()
```

Section 4 — Stage 2: Sentence Alignment & Excel Write

Reads each `transcript.txt`, discards the English practice section, fuzzy-matches segments to the 30 EIT stimuli, and writes column C of the Excel template. Missing sentences → `[no response]`.

Expected time: ~1 min for all 4 participants

```
!pip install rapidfuzz openpyxl langdetect
```

```
Requirement already satisfied: rapidfuzz in /usr/local/lib/python3.12/dist-packages (3.14.3)
Requirement already satisfied: openpyxl in /usr/local/lib/python3.12/dist-packages (3.1.5)
Requirement already satisfied: langdetect in /usr/local/lib/python3.12/dist-packages (1.0.9)
Requirement already satisfied: et-xmlfile in /usr/local/lib/python3.12/dist-packages (from openpyxl) (2.0.0)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from langdetect) (1.17.0)
```

4.1 — Imports, config & EIT stimuli

Defines:

- `PAIR_THRESHOLD` and `EXCEL_PATH`
- `EIT_STIMULI_ENGLISH` — the 6 English practice sentences used to detect the boundary
- `EIT_STIMULI_SPANISH` — all 30 Version A Spanish stimuli used for fuzzy matching

```
"""
AutoEIT Transcriber
———————————————————
Reads speaker_separator.py output (transcript.txt per file) and maps
each participant response to one of the 30 known EIT sentences, then
writes the result into column C of the Excel file.

Usage:
    python autoeit_transcriber.py
    python autoeit_transcriber.py --transcripts ./speaker_output --excel ./AutoEIT_Sample_Audio_for_Transcribing.xlsx

Requirements:
    pip install rapidfuzz openpyxl langdetect
"""

import os
import sys
import re
import argparse

try:
    from langdetect import detect as _langdetect
    _LANGDETECT_AVAILABLE = True
except ImportError:
    _LANGDETECT_AVAILABLE = False

# ─── CONFIG ──────────────────────────────────────────────────────────────

TRANSCRIPTS_ROOT = "./speaker_output"
EXCEL_PATH       = "./AutoEIT Sample Audio for Transcribing.xlsx"

# Threshold used to decide if a segment is a completely new attempt (OVERWRITE)
# or just a continuation of the previous transcript chunk (CONCATENATE).
PAIR_THRESHOLD = 55

# ─── THE 30 KNOWN EIT STIMULI ────────────────────────────────────────────

EIT_STIMULI_ENGLISH = [
    "We drove to the park",
    "I'll call her tomorrow night",
    "You can buy meat at the butcher shop",
    "My brother just bought a brand new computer",
    "Sometimes they take their dog for a walk in the park",
    "We're going to play volleyball at the gym that I told you about",
]

EIT_STIMULI_SPANISH = [
    "Quiero cortarme el pelo",
    "El libro está en la mesa",
    "El carro lo tiene Pedro",
    "Él se ducha cada mañana",
    "¿Qué dice usted que va a hacer hoy?",
    "Dudo que sepa manejar muy bien",
    "Las calles de esta ciudad son muy anchas",
    "Puede que llueva mañana todo el día",
    "Las casas son muy bonitas pero caras",
    "Me gustan las películas que acaban bien",
    "El chico con el que yo salgo es español",
    "Después de cenar me fui a dormir tranquilo",
    "Quiero una casa en la que vivan mis animales",
    "A nosotros nos fascinan las fiestas grandiosas",
    "Ella sólo bebe cerveza y no come nada",
    "Me gustaría que el precio de las casas bajara",
    "Cruza a la derecha y después sigue todo recto",
    "Ella ha terminado de pintar su apartamento",
    "Me gustaría que empezara a hacer más calor pronto",
    "El niño al que se le murió el gato está triste",
    "Una amiga mía cuida a los niños de mi vecino",
    "El gato que era negro fue perseguido por el perro",
    "Antes de poder salir él tiene que limpiar su cuarto",
```

```
    "La cantidad de personas que fuman ha disminuido",
    "Después de llegar a casa del trabajo tomé la cena",
    "El ladrón al que atrapó la policía era famoso",
    "Le pedí a un amigo que me ayudara con la tarea",
    "El examen no fue tan difícil como me habían dicho",
    "¿Serías tan amable de darme el libro que está en la mesa?",
    "Hay mucha gente que no toma nada para el desayuno",
]

```

4.2 — Helper functions & skip-phrase filter

- `normalize()` — strips punctuation and lowercases for fair fuzzy comparison
- `fuzzy_score()` — `token_sort_ratio` from rapidfuzz (word-order-independent similarity)
- `is_english()` — detects English text via langdetect (or keyword fallback)
- `SPANISH_INSTRUCTION_PHRASES` — EIT protocol instructions + Whisper prompt-leak fragments. Any segment matching these is silently discarded before alignment.

```

# ──────────────────────────────────────────────────────────────────────

COLORS = {
    "reset":  "\033[0m",  "bold":   "\033[1m",  "dim":    "\033[2m",
    "header": "\033[96m", "green":  "\033[92m",  "red":    "\033[91m",
    "yellow": "\033[93m", "blue":   "\033[94m",  "cyan":   "\033[96m",
}

def color(text, *keys):
    codes = "".join(COLORS.get(k, "") for k in keys)
    return f"{codes}{text}{COLORS['reset']}"

def print_header(title):
    print("\n" + color("─" * 70, "header"))
    print(color(f"  {title}", "header", "bold"))
    print(color("─" * 70, "header"))

def print_banner(title):
    print("\n" + color("=" * 70, "bold"))
    print(color(f"  {title}", "bold"))
    print(color("=" * 70, "bold"))


def check_requirements():
    missing = []
    for pkg, imp in [("rapidfuzz", "rapidfuzz"), ("openpyxl", "openpyxl")]:
        try:
            __import__(imp)
        except ImportError:
            missing.append(pkg)
    if missing:
        print(color(f"\n❌ Missing packages. Run:\n\n   pip install {' '.join(missing)}\n", "bold"))
        sys.exit(1)


def normalize(text):
    text = text.lower()
    text = re.sub(r"[¿¡.,!?;:\"'«»\-]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text


def fuzzy_score(a, b):
    from rapidfuzz import fuzz
    return fuzz.token_sort_ratio(normalize(a), normalize(b))


def best_stimulus_match(text, stimuli):
    """Return (best_stimulus, best_index_0based, best_score)."""
    best_score, best_idx, best_stim = -1, -1, ""
    for i, stim in enumerate(stimuli):
        score = fuzzy_score(text, stim)
        if score > best_score:
            best_score, best_idx, best_stim = score, i, stim
    return best_stim, best_idx, best_score


```

```
def is_english(text):
    """
    Return True if the segment is clearly English.
    Safety net to discard instruction/practice sentences
    even if boundary detection fails.
    Ignores segments < 2 words to avoid false positives.
    """
    if len(text.split()) < 2:
        return False
    if _LANGDETECT_AVAILABLE:
        try:
            return _langdetect(text) == "en"
        except Exception:
            return False
    # Fallback keyword check if langdetect not installed
    english_markers = {"the", "you", "your", "will", "please", "dont",
                       "after", "each", "until", "start", "hear", "tone",
                       "sentence", "repeat", "given", "sufficient", "careful",
                       "notes", "recording", "attention", "exercise", "bought",
                       "drive", "drove", "park", "call", "buy", "meat",
                       "butcher",
                       "brother", "computer", "sometimes", "volleyball", "gym"}
    words = set(normalize(text).split())
    return len(words & english_markers) >= 2


# ─── SKIP PHRASES ──────────────────────────────────────────────────────────
# Two categories:
#   1. EIT protocol instructions (Spanish + English)
#   2. Whisper prompt leak phrases — Whisper sometimes hallucinates the
#      initial_prompt text verbatim into a segment. Any segment containing
#      these substrings is ASR noise, not a participant response.

SPANISH_INSTRUCTION_PHRASES = [
    # ── EIT protocol instructions ──────────────────────────────────────
    "repite lo máximo que puedas",
    "repite lo más que puedas",
    "recuerda no comiences",
    "no comiences a repetir",
    "te darás suficiente tiempo",
    "darás suficiente tiempo",
    "después del tono para repetir",
    "escuches el tono",
    "hasta que escuches",
    "ahora volvemos",
    "ahora comenzamos",
    "vamos a comenzar",
    "ahora empezamos",
    "presta atención",
    "por favor no tomes",
    "no tomes notas",
    "no anotes nada",
    "now let s begin",
    "now let us begin",
    "let s begin",
    # ── Whisper prompt leak phrases ────────────────────────────────────
    # These are substrings from the INITIAL_PROMPT in speaker_separator.py.
    # If any appear in a segment it means Whisper hallucinated the prompt,
    # not a real participant response.
    "incluye errores gramaticales",
    "transcribe exactamente lo que dice",
    "falsos inicios con guion",
    "falsos inicios con guión",
    "no corrijas ni normalices",
    "no traduzcas ninguna frase",
    "si hay una pausa demasiado larga",
    "probablemente indica el inicio de la siguiente",
    "un estudiante de español l2 repite",
    "estudiante de español l2 repite una frase",
]


def should_skip(text):
    """
    Return True if a segment should be discarded entirely:
      - Clearly English (instruction / English practice section)
      - Matches a known EIT instruction phrase
      - Contains a Whisper prompt leak phrase
    """
    if is_english(text):
```

```
        return True
    text_norm = normalize(text)
    for phrase in SPANISH_INSTRUCTION_PHRASES:
        if phrase in text_norm:
            return True
    return False

```

4.3 — Transcript parsing & English/Spanish boundary detection

- `parse_transcript()` — reads `transcript.txt` into a list of `{start, end, text}` dicts
- `find_spanish_boundary()` — locates where the Spanish EIT begins using three fallback strategies:
  1. Last English/instruction segment within the first 35% of the recording
  2. Largest silence gap in the first 35%
  3. Absolute last English segment (final fallback)

```


def parse_transcript(transcript_path):
    """
    Parse transcript.txt → list of {start_sec, end_sec, text}.
    Handles lines like:  [02:34 → 02:37]  Speaker 1   El libro está...
    """
    segments = []
    pattern  = re.compile(r"\[(\d{2}:\d{2})\s*[→\-]+\s*(\d{2}:\d{2})\]\s*(.*)")

    def to_sec(t):
        mm, ss = t.split(":")
        return int(mm) * 60 + int(ss)

    with open(transcript_path, encoding="utf-8") as f:
        for line in f:
            m = pattern.search(line.strip())
            if not m:
                continue
            raw_text = m.group(3).strip()
            text     = re.sub(r"^(Speaker\s*\d+|Unknown)\s+", "", raw_text,
            flags=re.IGNORECASE).strip()
            if text:
                segments.append({
                    "start": to_sec(m.group(1)),
                    "end":   to_sec(m.group(2)),
                    "text":  text,
                })
    return segments


def find_spanish_boundary(segments):
    """
    Find the first segment index where the Spanish EIT trials start.
    """
    if len(segments) < 2:
        return 0

    total_duration = segments[-1]["end"]
    search_limit   = total_duration * 0.35

    # ── Priority 1: last English segment in first 35% ──────────────────
    last_english_idx = -1
    for i, seg in enumerate(segments):
        if seg["start"] > search_limit:
            break
        if is_english(seg["text"]) or should_skip(seg["text"]):
            last_english_idx = i

    if last_english_idx >= 0:
        boundary = last_english_idx + 1
        preview  = segments[last_english_idx]["text"][:50]
        ts       = segments[last_english_idx]["start"]
        print(f"    ✅ English/Spanish boundary: after segment #
        {last_english_idx} "
              f"(last English at {ts}s: '{preview}')")
        return boundary

    # ── Priority 2: largest silence gap in first 35% ──────────────────
    best_gap, best_idx = 0, 0
    for i in range(1, len(segments)):
```

```
        start_prev = segments[i-1]["start"]
        if start_prev < 60:
            continue
        if start_prev > search_limit:
            break
        gap = segments[i]["start"] - segments[i-1]["end"]
        if gap > best_gap:
            best_gap, best_idx = gap, i

    if best_idx > 0:
        print(f"    ✅ English/Spanish boundary: after segment #{best_idx} "
              f"(gap={best_gap:.0f}s  at {segments[best_idx]['start']}s)")
        return best_idx

    # ── Priority 3: fallback — absolute last English segment ──────────────
    last_absolute_english = -1
    for i, seg in enumerate(segments):
        if is_english(seg["text"]) or should_skip(seg["text"]):
            last_absolute_english = i

    if last_absolute_english >= 0:
        print(f"    ⚠️  Boundary fallback: using absolute last English segment
        (#{last_absolute_english})")
        return last_absolute_english + 1

    return 0

```

4.4 — Alignment engine

- `split_merged_segment()` — recovers two responses merged into one segment. Tries splitting on `...` first, then brute-forces all word boundaries looking for two halves that each match different stimuli
- `extract_responses()` — the core alignment loop. Uses **anchor gravity**: each stimulus gets a position bonus (12 / 8 / 4 pts) pulling weak matches toward the expected sentence number. Decision labels you'll see in output:
  - `ASSIGN` — solid match (score ≥ 40)
  - `GUESS` — weak match, uses position anchor
  - `CONCAT` — appended to previous response (improves the merged score)
  - `OVERWRITE` — better match than what was stored
  - `REPLACE (RESTART)` — participant restarted the test
  - `IGNORE (WEAK)` — discarded

```


def split_merged_segment(text, stimuli):
    """
    Detect and split segments containing TWO merged responses.
    """
    if "..." in text:
        parts_on_ellipsis = text.split("...")
        candidates = []
        for cut in range(1, len(parts_on_ellipsis)):
            left  = "...".join(parts_on_ellipsis[:cut]).strip().rstrip(".")
            right = "...".join(parts_on_ellipsis[cut:]).strip().lstrip(".")
            if len(left.split()) >= 2 and len(right.split()) >= 2:
                candidates.append((left, right))

        best_score, best_pair = 0, None
        _, _, single_score = best_stimulus_match(text, stimuli)

        for left, right in candidates:
            _, li, ls = best_stimulus_match(left,  stimuli)
            _, ri, rs = best_stimulus_match(right, stimuli)
            if li != ri and ls >= 40 and rs >= 40:
                combined = ls + rs
                if combined > best_score:
                    best_score, best_pair = combined, (left, right)

        if best_pair and best_score > single_score + 15:
            return list(best_pair)

    words = text.split()
    if len(words) < 8:
        return [text]

```

```
    _, _, single_score = best_stimulus_match(text, stimuli)
    best_combined, best_pair = 0, None

    for split in range(3, len(words) - 2):
        left  = " ".join(words[:split])
        right = " ".join(words[split:])
        _, li, ls = best_stimulus_match(left,  stimuli)
        _, ri, rs = best_stimulus_match(right, stimuli)

        if li != ri and ls >= 60 and rs >= 60:
            combined = ls + rs
            if combined > best_combined:
                best_combined, best_pair = combined, (left, right)

    if best_pair and best_combined > single_score + 30:
        return list(best_pair)

    return [text]


def extract_responses(segments, spanish_stimuli):
    responses = {}
    expected_idx = 0

    print_header("  DYNAMIC ALIGNMENT (WITH AUTO-CORRECT & RESTART DETECT)")

    for i, seg in enumerate(segments):
        if should_skip(seg["text"]):
            print(f"  {color('SKIP', 'dim')}  #{i:03d}  [filtered]  {seg['text'][:60]}")
            continue

        parts = split_merged_segment(seg["text"], spanish_stimuli)

        for part in parts:
            best_idx  = -1
            best_score = -1

            for j, stim in enumerate(spanish_stimuli):
                base_score = fuzzy_score(part, stim)
                bonus = 0
                if j == expected_idx:     bonus = 12
                elif j == expected_idx+1: bonus = 8
                elif j == expected_idx+2: bonus = 4
                score = base_score + bonus
                if score > best_score:
                    best_score = score
                    best_idx   = j

            actual_fuzzy = fuzzy_score(part, spanish_stimuli[best_idx])

            if actual_fuzzy >= 40:
                target_idx   = best_idx
                expected_idx = target_idx + 1
            else:
                target_idx = expected_idx
                if expected_idx > 0:
                    prev_idx = expected_idx - 1
                    old_text = responses.get(prev_idx + 1, "")
                    merged   = old_text + " " + part
                    if fuzzy_score(merged, spanish_stimuli[prev_idx]) > fuzzy_score(old_text, spanish_stimuli[prev_idx
                        target_idx   = prev_idx
                        expected_idx = prev_idx + 1

            if target_idx >= 30:
                print(f"  {color('DROP (END)', 'dim')}  #{i:03d}  [past sentence 30] {part[:40]}...")
                continue

            sentence_num = target_idx + 1
            stimulus     = spanish_stimuli[target_idx]

            if sentence_num in responses:
                old_text     = responses[sentence_num]
                score_old    = fuzzy_score(old_text, stimulus)
                score_new    = fuzzy_score(part, stimulus)
                merged       = old_text + " " + part
                score_merged = fuzzy_score(merged, stimulus)

                if score_merged > max(score_old, score_new) + 2:
                    responses[sentence_num] = merged
                    print(f"  {color('CONCAT', 'yellow')}  #{i:03d}  →  sentence {sentence_num:02d}")
```

```
                elif score_new > score_old + 3:
                    responses[sentence_num] = part
                    print(f"  {color('OVERWRITE', 'green')}  #{i:03d}  →  sentence {sentence_num:02d} (improved match)
                elif actual_fuzzy >= 40 and score_new >= score_old - 5:
                    responses[sentence_num] = part
                    print(f"  {color('REPLACE (RESTART)', 'cyan')}  #{i:03d}  →  sentence {sentence_num:02d}")
                else:
                    print(f"  {color('IGNORE (WEAK)', 'dim')}  #{i:03d}  [ignored for {sentence_num:02d}] {part[:40]}.
            else:
                responses[sentence_num] = part
                tag = 'ASSIGN' if actual_fuzzy >= 40 else 'GUESS'
                col = 'blue'   if actual_fuzzy >= 40 else 'dim'
                print(f"  {color(tag, col)}  #{i:03d}  →  sentence {sentence_num:02d} (score: {actual_fuzzy:.1f})")

            if target_idx == expected_idx and actual_fuzzy < 40:
                expected_idx += 1

    return responses

```

4.5 — Excel writer & transcript processor

- `write_to_excel()` — finds the participant's sheet by ID substring match, writes detected responses to column C, fills `[no response]` for any missing sentence 1–30
- `process_transcript()` — orchestrates everything: parse → find boundary → extract → write

```


def write_to_excel(excel_path, file_id, responses):
    """
    Write responses into column C of the matching participant sheet.
    Any sentence number with no detected response gets '[no response]'.
    """
    import openpyxl
    wb = openpyxl.load_workbook(excel_path)

    # Find the sheet for this participant
    target_sheet = None
    for sheet_name in wb.sheetnames:
        if str(file_id) in sheet_name:
            target_sheet = wb[sheet_name]
            break

    if target_sheet is None:
        print(color(f"    ⚠️  No sheet found containing ID '{file_id}'. Skipping write.", "yellow"))
        return 0

    filled   = 0
    no_resp  = 0

    for row in target_sheet.iter_rows(min_row=2):
        cell_a = row[0].value   # Column A: sentence number

        try:
            sentence_num = int(cell_a)
        except (ValueError, TypeError):
            continue

        if sentence_num < 1 or sentence_num > 30:
            continue

        if sentence_num in responses:
            # ── Response detected — write it ──────────────────────────
            row[2].value = responses[sentence_num]
            filled += 1
        else:
            # ── No response detected — write [no response] ────────────
            row[2].value = "[no response]"
            no_resp += 1

    wb.save(excel_path)
    print(f"    ✅ Wrote {filled}/30 responses  +  {no_resp} × '[no response]'  →  sheet '{target_sheet.title}'")
    return filled + no_resp


def process_transcript(transcript_path, file_id, excel_path):
    print(color(f"\n  📄 Parsing: {transcript_path}", "dim"))
```

```
    segments = parse_transcript(transcript_path)
    print(f"    ✅ {len(segments)} segments loaded")

    if not segments:
        print(color("    ⚠️  No segments found — skipping", "yellow"))
        return {}

    boundary = find_spanish_boundary(segments)
    spanish  = segments[boundary:]
    print(f"    ✅ {len(spanish)} segments after boundary")

    responses = extract_responses(spanish, EIT_STIMULI_SPANISH)

    print_header(f"  RESPONSES EXTRACTED  ({len(responses)}/30)")
    for n in range(1, 31):
        resp = responses.get(n, color("— not found → [no response]", "yellow"))
        num  = color(f"  {n:02d}.", "dim")
        stim = color(f"  [{EIT_STIMULI_SPANISH[n-1][:45]}]", "dim")
        print(f"{num}{stim}")
        print(f"      → {resp}")

    if os.path.isfile(excel_path):
        print_header("  WRITING TO EXCEL")
        write_to_excel(excel_path, file_id, responses)
    else:
        print(color(f"\n  ⚠️  Excel not found at {excel_path} — skipping write", "yellow"))

    return responses


def infer_participant_id(folder_name):
    m = re.match(r"(\d+)", os.path.basename(folder_name))
    return str(int(m.group(1))) if m else folder_name

```

4.6 — `main()` — run on all transcripts ▶️

Scans `./output/` for `transcript.txt` files, infers participant ID from folder name, and runs the full alignment pipeline for each. **Run this cell to start Stage 2.**

```


def main():
    check_requirements()

    transcripts_root = "./output"
    excel_path       = "./AutoEIT Sample Audio for Transcribing.xlsx"

    if not os.path.isdir(transcripts_root):
        print(color(f"\n❌ Transcripts folder not found: {transcripts_root}\n", "bold"))
        sys.exit(1)

    transcript_files = []
    for folder in sorted(os.listdir(transcripts_root)):
        folder_path = os.path.join(transcripts_root, folder)
        tx_path     = os.path.join(folder_path, "transcript.txt")
        if os.path.isdir(folder_path) and os.path.isfile(tx_path):
            transcript_files.append((folder, tx_path))

    if not transcript_files:
        print(color(f"\n⚠️  No transcript.txt files found in: {transcripts_root}\n", "yellow"))
        sys.exit(0)

    print_banner("AutoEIT TRANSCRIBER")
    print(f"\n  📁 Transcripts : {transcripts_root}/")
    print(f"  📊 Excel       : {excel_path}")
    print(f"  📄 Found       : {len(transcript_files)} transcript(s)\n")

    summary = []
    for folder_name, tx_path in transcript_files:
        participant_id = infer_participant_id(folder_name)
        print_banner(f"{folder_name}  (ID: {participant_id})")
        try:
            responses = process_transcript(tx_path, participant_id, excel_path)
            summary.append((folder_name, "✅", len(responses)))
        except Exception as e:
            print(color(f"\n  ❌ Failed: {e}", "red", "bold"))
```

```
            summary.append((folder_name, "❌", 0))

    print_banner("COMPLETE 🎉")
    print(f"\n  {'FILE':<35}  {'STATUS':<6}  RESPONSES")
    print(f"  {'-'*35}  {'-'*6}  {'-'*10}")
    for fname, status, n in summary:
        print(f"  {fname:<35}  {status:<6}  {n}/30")

    passed = sum(1 for _, s, _ in summary if s == "✅")
    print(f"\n  {passed}/{len(summary)} files processed successfully.\n")


if __name__ == "__main__":
    main()
```

```
  ═══════════════════════════════════════════════════════════════════
  AutoEIT TRANSCRIBER
  ═══════════════════════════════════════════════════════════════════

  📁 Transcripts : ./output/
  📊 Excel       : ./AutoEIT Sample Audio for Transcribing.xlsx
  📄 Found       : 4 transcript(s)


  ═══════════════════════════════════════════════════════════════════
  038010_EIT-2A  (ID: 38010)
  ═══════════════════════════════════════════════════════════════════

  📄 Parsing: ./output/038010_EIT-2A/transcript.txt
    ✅ 54 segments loaded
    ✅ English/Spanish boundary: after segment #23 (last English at 158s: 'Now let's begin.')
    ✅ 30 segments after boundary

  ───────────────────────────────────────────────────────────────────
    DYNAMIC ALIGNMENT (WITH AUTO-CORRECT & RESTART DETECT)
  ───────────────────────────────────────────────────────────────────
  ASSIGN  #000  →  sentence 01 (score: 100.0)
  REPLACE (RESTART)  #001  →  sentence 01
  ASSIGN  #002  →  sentence 02 (score: 100.0)
  ASSIGN  #003  →  sentence 03 (score: 88.9)
  ASSIGN  #004  →  sentence 04 (score: 87.0)
  ASSIGN  #005  →  sentence 05 (score: 100.0)
  ASSIGN  #006  →  sentence 06 (score: 100.0)
  ASSIGN  #007  →  sentence 07 (score: 100.0)
  ASSIGN  #008  →  sentence 08 (score: 100.0)
  ASSIGN  #009  →  sentence 09 (score: 100.0)
  ASSIGN  #010  →  sentence 10 (score: 100.0)
  ASSIGN  #011  →  sentence 11 (score: 71.6)
  ASSIGN  #012  →  sentence 12 (score: 100.0)
  IGNORE (WEAK)  #013  [ignored for 12] Después de cenar, me fui a dormir tranqu...
  ASSIGN  #013  →  sentence 13 (score: 85.7)
  ASSIGN  #014  →  sentence 15 (score: 87.1)
  OVERWRITE  #015  →  sentence 15 (improved match)
  GUESS  #016  →  sentence 16 (score: 39.3)
  ASSIGN  #016  →  sentence 17 (score: 100.0)
  IGNORE (WEAK)  #017  [ignored for 17] Cruza a la derecha y después sigue recto...
  ASSIGN  #018  →  sentence 18 (score: 88.3)
  ASSIGN  #019  →  sentence 20 (score: 89.2)
  ASSIGN  #020  →  sentence 21 (score: 97.7)
  IGNORE (WEAK)  #021  [ignored for 21] Una amiga mía cuidó a los niños....
  ASSIGN  #022  →  sentence 22 (score: 60.0)
  ASSIGN  #022  →  sentence 23 (score: 97.0)
  ASSIGN  #023  →  sentence 24 (score: 61.5)
  ASSIGN  #023  →  sentence 26 (score: 100.0)
  IGNORE (WEAK)  #024  [ignored for 26] El ladrón policía era famoso....
  ASSIGN  #025  →  sentence 27 (score: 100.0)
  ASSIGN  #026  →  sentence 28 (score: 60.0)
  ASSIGN  #027  →  sentence 29 (score: 83.0)
  IGNORE (WEAK)  #027  [ignored for 02] está en la mesa?...
  IGNORE (WEAK)  #028  [ignored for 29] ¿Amable que está el libro que...
  IGNORE (WEAK)  #028  [ignored for 02] está en la mesa?...
```

Section 5 — Download Outputs

Zips the `./output/` folder and downloads the filled Excel submission file.

```
import shutil

# Create a zip file from the output folder
shutil.make_archive("output", "zip", "./output")
```

```
'/content/output.zip'
```

```python
# Download submission Excel + transcript zip
from google.colab import files
import shutil
from pathlib import Path

src  = Path("./AutoEIT Sample Audio for Transcribing.xlsx")
dest = Path("./AutoEIT_Sample_Audio_for_Transcribing_FILLED.xlsx")
if src.exists():
    shutil.copy(src, dest)
    print(f"✅ {dest.name}")
    files.download(str(dest))
files.download("output.zip")
```

```
✅ AutoEIT_Sample_Audio_for_Transcribing_FILLED.xlsx
```

Section 6 — Utility: Extract Output Zip

Restores `./output/` from a previously saved zip.

```python
import zipfile

with zipfile.ZipFile("output.zip", 'r') as zip_ref:
    zip_ref.extractall("./output")
```